

Coding and Software Cookbook

Data Analysis Companion v3

Python and R workflow companion for the Data Analysis textbook

Contents

Coding and Software Cookbook	1
1 How to Use This Cookbook	1
2 Cookbook Operating Rules	2
3 Project Setup and Reproducibility	3
4 General Setup Patterns	4
5 Data Quality and Cleaning	5
5.1 Chapter 3 Support: Data-Quality First Pass	5
5.2 True Zeros, Placeholder Zeros, and Missingness Flags	6
5.3 Dropping, Imputing, and Flagging Missing Values	7
5.4 Fixing Data Types and Standardizing Columns	8
5.5 Impossible Values, Outlier Flags, and Cleaned Data	9
6 EDA Summaries and Visuals	10
6.1 Chapter 4 Support: EDA Summaries and Visuals	10
7 Simple Linear Regression Workflow	12
7.1 Chapter 5 Support: Fitting Simple Linear Regression	12
7.2 Chapter 6 Support: Fit Quality and Hypothesis Testing	14
8 Regression Geometry and Design Matrices	15
8.1 Chapter 7 Support: Design Matrix and Hat Matrix	15
8.2 Chapter 8 Support: Degrees of Freedom and Identifiability	17
9 Multiple Linear Regression Workflow	18
9.1 Chapter 9 Support: Multiple Linear Regression in Practice	18
10 Estimator Behavior and Residual Objects	21
10.1 Chapter 10 Support: Python Residual and Coefficient Objects	21
10.2 Chapter 10 Support: R Residual and Coefficient Objects	22
11 Transformations and Linear-in-Parameters Models	23
11.1 Chapter 11 Support: Python Transformations	23
11.2 Chapter 11 Support: R Transformations	24

12	Residual Diagnostics and Model Repair	25
12.1	Chapter 12 Support: Python Model Diagnostics and Repair	25
12.2	Chapter 12 Support: R Model Diagnostics and Repair	27
13	End-to-End Mini Workflow	28
14	Quick Troubleshooting Guide	31
15	Cookbook Habits	31

Coding and Software Cookbook

This companion collects implementation patterns that support the Data Analysis textbook. The main chapters develop concepts, assumptions, interpretation, and mathematical structure. This cookbook keeps the package-specific Python and R workflows close at hand so students can turn those concepts into reproducible analyses during labs and assignments.

The cookbook is organized by analytic workflow rather than by lecture order. Each block names the chapter material it supports, but the recipes are meant to be reused across projects. Code is not a substitute for judgment: it can compute summaries, fit models, and extract diagnostics, but the analyst still decides whether the result is meaningful and defensible.

1 How to Use This Cookbook

Use this companion after reading the corresponding chapter or while working through a lab.

- Start with the question: What are you trying to learn, predict, or explain?
- Identify the response, predictors, units, and variable types before running a model.
- Use each code block as a pattern. Rename variables, check units, and verify assumptions before reusing it.
- Inspect output before reporting it. A model summary is not an analysis until you interpret it.
- Preserve enough code and notes that another analyst can reproduce the result.

Workflow block	Main chapter support	Main purpose
Project setup	Chapters 1–3	Organize a reproducible project before analysis begins
Data quality and cleaning	Chapter 3	Inspect columns, missingness, placeholder values, data types, impossible values, and cleaned-data outputs
EDA patterns	Chapter 4	Compute summaries, create plots, and inspect correlations before modeling
SLR fitting	Chapter 5	Fit simple regression, compute coefficients manually, and extract residuals
SLR fit and inference	Chapter 6	Compute R^2 , F -statistics, p-values, RSS, TSS, and related output
Regression geometry	Chapters 7–8	Build design matrices, hat matrices, leverage values, rank checks, and residual degrees of freedom
MLR practice	Chapter 9	Fit full/reduced multiple-regression models, predict, compare fit, and plot residuals
Residual objects	Chapter 10	Extract fitted values, residuals, leverage, studentized residuals, coefficient covariance, and diagnostic plots
Transformations	Chapter 11	Fit polynomial, interaction, log-response, and linear-in-parameters models
Model repair	Chapter 12	Diagnose residuals, compare repaired models, and validate on a common response scale

2 Cookbook Operating Rules

Every recipe in this companion should be used under the same operating rules.

1. **Start from the analytic question.** A command is only useful if it answers a question that matters.
2. **Declare the response and predictors.** Write down what each column means before modeling.
3. **Check variable types before summarizing or modeling.** A numeric-looking column may be a category code or an identifier.
4. **Clean before modeling, but document the cleaning.** Record what was dropped, recoded, imputed, or flagged.
5. **Split before learned preprocessing.** If using train/test validation, compute learned transformations from the training set only.
6. **Fit the simplest defensible model first.** More complicated repairs should be motivated by residual behavior, validation error, or domain knowledge.
7. **Extract diagnostics, not only coefficients.** Fitted values, residuals, leverage, and prediction error are part of the model output.

8. **Interpret before reporting.** Report what the output means for the original problem, including uncertainty and limitations.

3 Project Setup and Reproducibility

Purpose. Use a consistent project structure so the analysis can be rerun and audited.

A simple course-project folder can look like this:

```
project_name/  
  README.md  
  data/  
    raw/  
    cleaned/  
  notebooks/  
  scripts/  
  outputs/  
    figures/  
    tables/  
  reports/
```

Defensibility check. A reviewer should be able to answer: Where did the raw data come from? What cleaning decisions were made? Which file produces each figure or table?

Python: create common project folders

```
from pathlib import Path  
  
project = Path("project_name")  
for folder in [  
    "data/raw",  
    "data/cleaned",  
    "notebooks",  
    "scripts",  
    "outputs/figures",  
    "outputs/tables",  
    "reports"  
]:  
    (project / folder).mkdir(parents=True, exist_ok=True)
```

R: create common project folders

```
folders <- c(  
  "project_name/data/raw",  
  "project_name/data/cleaned",  
  "project_name/notebooks",  
)
```

```

    "project_name/scripts",
    "project_name/outputs/figures",
    "project_name/outputs/tables",
    "project_name/reports"
)

for (folder in folders) {
  dir.create(folder, recursive = TRUE, showWarnings = FALSE)
}

```

README pattern

A short README.md should include:

```

# Project Name

## Question
What analytic question is this project answering?

## Data
Where did the raw data come from? When was it accessed?

## Workflow
1. Load raw data
2. Clean and validate variables
3. Run EDA
4. Fit models
5. Diagnose and compare models
6. Report conclusions

## Reproducibility
Which script or notebook reproduces the main results?

```

4 General Setup Patterns

Most Python examples use the following package pattern.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

```

Regression examples that require inference usually use `statsmodels`.

```

import statsmodels.api as sm
import statsmodels.formula.api as smf

```

Prediction-focused examples often use `sklearn`.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

A version-stable RMSE pattern is:

```
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
```

In R, most regression examples begin with `lm()`.

```
fit <- lm(y ~ x1 + x2, data = df)
summary(fit)
```

5 Data Quality and Cleaning

5.1 Chapter 3 Support: Data-Quality First Pass

Purpose. Inspect a data table before summary statistics, plots, or models. This recipe answers: What columns exist, what types did the software infer, and where are the obvious missing or impossible values?

Python pattern

```
import pandas as pd

# Load raw data
raw_path = "data/raw/my_data.csv"
df = pd.read_csv(raw_path)

# Basic structure
print(df.shape)
print(df.head())
print(df.info())

# Column names and inferred types
print(df.columns.tolist())
print(df.dtypes)

# Missing counts and rates
missing_count = df.isna().sum()
missing_rate = df.isna().mean()
missing_summary = pd.DataFrame({
    "missing_count": missing_count,
    "missing_rate": missing_rate
}).sort_values("missing_rate", ascending=False)
```



```
print(missing_summary)

# Duplicate rows
print("duplicate rows:", df.duplicated().sum())
```

R pattern

```
# Load raw data
raw_path <- "data/raw/my_data.csv"
df <- read.csv(raw_path)

# Basic structure
dim(df)
head(df)
str(df)

# Missing counts and rates
missing_count <- colSums(is.na(df))
missing_rate <- colMeans(is.na(df))
missing_summary <- data.frame(
  missing_count = missing_count,
  missing_rate = missing_rate
)
missing_summary[order(-missing_summary$missing_rate), ]

# Duplicate rows
sum(duplicated(df))
```

Output to inspect. Row count, column count, column names, inferred data types, missingness rates, and duplicate rows.

Defensibility check. Can you explain whether each column is numeric, ordinal, categorical, date/time, text, identifier, or derived?

5.2 True Zeros, Placeholder Zeros, and Missingness Flags

Purpose. Distinguish valid zeros from placeholder zeros. A zero in `sibling_count` may be meaningful; a zero in `weight_lb` is usually invalid.

Python pattern

```
# Example: zero is not physically plausible for weight_lb
bad_weight_zero = df["weight_lb"] == 0
print("weight zero count:", bad_weight_zero.sum())

# Create a missingness flag before replacing values
```

```
df["weight_lb_was_zero"] = bad_weight_zero

# Replace placeholder zero with missing value
df.loc[bad_weight_zero, "weight_lb"] = np.nan

# Example: zero is valid for sibling_count
print(df["sibling_count"].value_counts(dropna=False).sort_index())
```

R pattern

```
# Example: zero is not physically plausible for weight_lb
bad_weight_zero <- df$weight_lb == 0
sum(bad_weight_zero, na.rm = TRUE)

# Create a missingness flag before replacing values
df$weight_lb_was_zero <- bad_weight_zero

# Replace placeholder zero with missing value
df$weight_lb[bad_weight_zero] <- NA

# Example: zero is valid for sibling_count
table(df$sibling_count, useNA = "ifany")
```

Common mistake. Do not run a universal “replace all zeros with missing” rule. The decision depends on the variable definition.

5.3 Dropping, Imputing, and Flagging Missing Values

Purpose. Implement the Chapter 3 decision workflow: drop only when justified, impute only when the assumption is defensible, and flag missingness when the fact of missingness may matter.

Python pattern

```
# Drop rows missing a required response variable
model_df = df.dropna(subset=["y"])

# Drop rows missing key predictors for a complete-case model
complete_df = model_df.dropna(subset=["x1", "x2"])

# Simple median imputation for a numerical predictor
x1_median = model_df["x1"].median()
model_df["x1_missing"] = model_df["x1"].isna()
model_df["x1_imputed"] = model_df["x1"].fillna(x1_median)

# Mode imputation for a categorical variable
mode_value = model_df["group"].mode(dropna=True).iloc[0]
```

```
model_df["group_missing"] = model_df["group"].isna()
model_df["group_imputed"] = model_df["group"].fillna(mode_value)
```

R pattern

```
# Drop rows missing a required response variable
model_df <- df[!is.na(df$y), ]

# Drop rows missing key predictors for a complete-case model
complete_df <- model_df[complete.cases(model_df[, c("x1", "x2")]), ]

# Simple median imputation for a numerical predictor
x1_median <- median(model_df$x1, na.rm = TRUE)
model_df$x1_missing <- is.na(model_df$x1)
model_df$x1_imputed <- ifelse(is.na(model_df$x1), x1_median, model_df$x1)

# Mode imputation for a categorical variable
mode_value <- names(sort(table(model_df$group), decreasing = TRUE))[1]
model_df$group_missing <- is.na(model_df$group)
model_df$group_imputed <- ifelse(
  is.na(model_df$group),
  mode_value,
  model_df$group
)
```

Output to inspect. Number of rows lost, missingness flags, imputed value used, and whether distributions changed after imputation.

Defensibility check. Be able to say: “This value was imputed because ...” or “This row was removed because ...”. Imputation is an assumption, not a fact.

5.4 Fixing Data Types and Standardizing Columns

Purpose. Convert columns to types that match their analytical meaning and standardize names so later code is easier to audit.

Python pattern

```
# Standardize column names
clean_names = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_", regex=False)
    .str.replace("-", "_", regex=False)
)
```

```

df.columns = clean_names

# Convert numeric stored as text
df["height_in"] = pd.to_numeric(df["height_in"], errors="coerce")

# Convert categorical code to category, not numeric magnitude
df["service_branch"] = df["service_branch"].astype("category")

# Convert dates stored as text
df["date"] = pd.to_datetime(df["date"], errors="coerce")

```

R pattern

```

# Standardize column names
names(df) <- tolower(trimws(names(df)))
names(df) <- gsub(" ", "_", names(df))
names(df) <- gsub("-", "_", names(df))

# Convert numeric stored as text
df$height_in <- as.numeric(df$height_in)

# Convert categorical code to factor, not numeric magnitude
df$service_branch <- factor(df$service_branch)

# Convert dates stored as text
df$date <- as.Date(df$date)

```

Common mistake. Do not average category codes such as Army = 1, Marine Corps = 2, Navy = 3. Those are labels, not numerical amounts.

5.5 Impossible Values, Outlier Flags, and Cleaned Data

Purpose. Flag impossible values and save a cleaned data set with documented decisions.

Python pattern

```

# Flag impossible values
checks = {
    "height_nonpositive": df["height_in"] <= 0,
    "weight_nonpositive": df["weight_lb"] <= 0,
    "age_negative": df["age"] < 0
}

for name, mask in checks.items():
    print(name, mask.sum())

```

```

# Tukey outlier flag for a numeric column
q1 = df["score"].quantile(0.25)
q3 = df["score"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

df["score_outlier_flag"] = (df["score"] < lower) | (df["score"] > upper)

# Save cleaned data
df.to_csv("data/cleaned/my_data_cleaned.csv", index=False)

```

R pattern

```

# Flag impossible values
sum(df$height_in <= 0, na.rm = TRUE)
sum(df$weight_lb <= 0, na.rm = TRUE)
sum(df$age < 0, na.rm = TRUE)

# Tukey outlier flag for a numeric column
q1 <- quantile(df$score, 0.25, na.rm = TRUE)
q3 <- quantile(df$score, 0.75, na.rm = TRUE)
iqr <- q3 - q1
lower <- q1 - 1.5 * iqr
upper <- q3 + 1.5 * iqr

df$score_outlier_flag <- df$score < lower | df$score > upper

# Save cleaned data
write.csv(df, "data/cleaned/my_data_cleaned.csv", row.names = FALSE)

```

Defensibility check. Outlier flags are not automatic deletion rules. If an observation is removed, record why.

6 EDA Summaries and Visuals

6.1 Chapter 4 Support: EDA Summaries and Visuals

Purpose. Compute summaries and plots that reveal center, spread, shape, extremes, and relationships among variables.

Python: summaries and basic plots

```

import pandas as pd
import numpy as np

```

```

import matplotlib.pyplot as plt

# Basic numerical summaries
df["score"].describe()

# Center and spread
mean_score = df["score"].mean()
median_score = df["score"].median()
sd_score = df["score"].std()
iqr_score = df["score"].quantile(0.75) - df["score"].quantile(0.25)

# Histogram with mean and median lines
ax = df["score"].hist(bins=20)
ax.axvline(mean_score, linestyle="--", label="mean")
ax.axvline(median_score, linestyle=":", label="median")
ax.legend()

# Grouped box plot
ax = df.boxplot(column="score", by="group")

# Correlation matrix
numeric_cols = ["score", "height_in", "weight_lb"]
corr = df[numeric_cols].corr()

```

R: summaries and basic plots

```

# Basic numerical summaries
summary(df$score)

# Center and spread
mean_score <- mean(df$score, na.rm = TRUE)
median_score <- median(df$score, na.rm = TRUE)
sd_score <- sd(df$score, na.rm = TRUE)
iqr_score <- IQR(df$score, na.rm = TRUE)

# Histogram with mean and median lines
hist(df$score)
abline(v = mean_score, lty = 2)
abline(v = median_score, lty = 3)

# Grouped box plot
boxplot(score ~ group, data = df)

# Correlation matrix
numeric_cols <- c("score", "height_in", "weight_lb")
cor(df[numeric_cols], use = "complete.obs")

```

Python: heatmap pattern

```
import seaborn as sns

corr = df[numeric_cols].corr()
sns.heatmap(corr, annot=True, center=0)
```

Output to inspect. Distributions, impossible values, skewness, unusually large spread, grouped differences, and strong pairwise correlations.

Defensibility check. A plot should answer a question. State what pattern the plot reveals and what follow-up it suggests.

7 Simple Linear Regression Workflow

7.1 Chapter 5 Support: Fitting Simple Linear Regression

Purpose. Fit an SLR model, extract coefficients, compute residuals, and create basic model-check plots.

Python: fit SLR with statsmodels

```
import pandas as pd
import statsmodels.api as sm

# df has columns: response y and predictor x
y = df["y"]
X = sm.add_constant(df["x"]) # adds intercept column

model = sm.OLS(y, X).fit()
print(model.summary())

beta0_hat = model.params["const"]
beta1_hat = model.params["x"]
residuals = model.resid
fitted = model.fittedvalues
rss = (residuals ** 2).sum()
mse = (residuals ** 2).mean()
```

Python: compute the least squares formulas manually

```
x = df["x"]
y = df["y"]

xbar = x.mean()
```

```

ybar = y.mean()

beta1_hat = ((x - xbar) * (y - ybar)).sum() / ((x - xbar) ** 2).sum()
beta0_hat = ybar - beta1_hat * xbar

print(beta0_hat, beta1_hat)

```

Python: quick scatterplot and residual plot

```

import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.scatter(df["x"], df["y"])
ax.plot(df["x"], beta0_hat + beta1_hat * df["x"])
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Simple linear regression fit")

fig, ax = plt.subplots()
ax.scatter(fitted, residuals)
ax.axhline(0, linestyle="--")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.set_title("Residuals versus fitted values")

```

R: fit SLR with lm

```

# df has columns: y and x
fit <- lm(y ~ x, data = df)
summary(fit)

beta0_hat <- coef(fit)[1]
beta1_hat <- coef(fit)[2]
residuals <- resid(fit)
fitted_values <- fitted(fit)
rss <- sum(residuals^2)
mse <- mean(residuals^2)

```

R: manual formula

```

x <- df$x
y <- df$y

xbar <- mean(x)
ybar <- mean(y)

beta1_hat <- sum((x - xbar) * (y - ybar)) / sum((x - xbar)^2)

```



```
beta0_hat <- ybar - beta1_hat * xbar
```

Common mistake. Coefficients are not automatically causal. They describe fitted associations unless the data collection and design justify a causal interpretation.

7.2 Chapter 6 Support: Fit Quality and Hypothesis Testing

Purpose. Extract fit quality and statistical-evidence quantities from an SLR model.

Python: SLR with statsmodels

```
import pandas as pd
import statsmodels.api as sm

# df has columns y and x
y = df["y"]
X = sm.add_constant(df["x"])

fit = sm.OLS(y, X).fit()
print(fit.summary())

r2 = fit.rsquared
f_stat = fit.fvalue
f_pvalue = fit.f_pvalue
residuals = fit.resid
fitted = fit.fittedvalues
rss = (residuals ** 2).sum()
tss = ((y - y.mean()) ** 2).sum()
manual_r2 = 1 - rss / tss

print(r2, f_stat, f_pvalue, manual_r2)
```

Python: manual R^2 and F

```
n = len(y)
rss = ((y - fitted) ** 2).sum()
tss = ((y - y.mean()) ** 2).sum()
r2 = 1 - rss / tss
F = (tss - rss) / (rss / (n - 2))
```

R: SLR with lm

```
fit <- lm(y ~ x, data = df)
summary(fit)

r2 <- summary(fit)$r.squared
```

```
f_stat <- summary(fit)$fstatistic
residuals <- resid(fit)
fitted_values <- fitted(fit)
rss <- sum(residuals^2)
tss <- sum((df$y - mean(df$y))^2)
manual_r2 <- 1 - rss / tss
```

R: manual F p-value

```
n <- nrow(df)
F_obs <- (tss - rss) / (rss / (n - 2))
p_value <- pf(F_obs, df1 = 1, df2 = n - 2, lower.tail = FALSE)
```

Output to inspect. R^2 , RSS, TSS, slope estimate, slope p-value, overall F -statistic, and residual plot.

Defensibility check. Separate fit quality from statistical evidence. A high R^2 is not the same as proof of a useful or causal model.

8 Regression Geometry and Design Matrices

8.1 Chapter 7 Support: Design Matrix and Hat Matrix

Purpose. Build the design matrix, compute the hat matrix, and verify the projection identities used in the textbook.

Python: manual hat matrix

The direct inverse below is included to make the projection geometry visible. For routine modeling, prefer `statsmodels`, `lm()`, or a numerically stable least-squares solver rather than manually inverting $\mathbf{X}^T\mathbf{X}$.

```
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
y = np.array([1.0, 0.0, 2.0])

X = np.column_stack([np.ones(len(x)), x])
beta_hat = np.linalg.inv(X.T @ X) @ X.T @ y
H = X @ np.linalg.inv(X.T @ X) @ X.T

y_hat = H @ y
e = y - y_hat

print(beta_hat)
```

```

print(y_hat)
print(e)
print(X.T @ e)      # should be close to zero
print(H.T - H)      # should be close to zero
print(H @ H - H)    # should be close to zero

```

Python: with statsmodels

```

import statsmodels.api as sm

X_sm = sm.add_constant(x)
fit = sm.OLS(y, X_sm).fit()

beta_hat = fit.params
y_hat = fit.fittedvalues
residuals = fit.resid

influence = fit.get_influence()
h_ii = influence.hat_matrix_diag

```

R: manual hat matrix

```

x <- c(-1, 0, 1)
y <- c(1, 0, 2)

X <- cbind(1, x)
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y
H <- X %*% solve(t(X) %*% X) %*% t(X)

y_hat <- H %*% y
e <- y - y_hat

t(X) %*% e      # should be zero
H %*% H - H     # should be zero

```

R: with lm

```

fit <- lm(y ~ x)
coef(fit)
fitted(fit)
resid(fit)
hatvalues(fit)

```

Common mistake. Do not confuse projection in $\mathbb{R}^n$ observation space with drawing a line on the two-dimensional scatterplot. The fitted vector $\hat{\mathbf{y}}$ is the projection object.

8.2 Chapter 8 Support: Degrees of Freedom and Identifiability

Purpose. Check rank, residual degrees of freedom, leverage sums, and non-identifiability.

Python: check rank and residual degrees of freedom

```
import numpy as np

# Example design matrix with intercept and two predictors
x1 = np.array([1, 2, 3, 4, 5], dtype=float)
x2 = np.array([2, 1, 3, 5, 4], dtype=float)
y = np.array([3, 4, 5, 7, 8], dtype=float)

X = np.column_stack([np.ones(len(x1)), x1, x2])
n, k = X.shape          # k = p + 1 if an intercept is included
r = np.linalg.matrix_rank(X)

print("n:", n)
print("columns k:", k)
print("rank:", r)
print("fitted-space df:", r)
print("residual df:", n - r)
print("full column rank?", r == k)
```

Python: detect a non-identifiable temperature design

```
import numpy as np

C = np.array([0, 10, 20, 30, 40], dtype=float)
F = 9/5 * C + 32

X = np.column_stack([np.ones(len(C)), C, F])

print(np.linalg.matrix_rank(X)) # rank is 2, but X has 3 columns
print(X @ np.array([-32, -9/5, 1.0])) # should be zero
```

Python: null space from SVD

```
import numpy as np

U, s, Vt = np.linalg.svd(X)
rank = np.linalg.matrix_rank(X)
null_basis = Vt[rank:].T

print(null_basis)
```

Python: statsmodels residual degrees of freedom

```
import statsmodels.api as sm

X_sm = sm.add_constant(np.column_stack([x1, x2]))
fit = sm.OLS(y, X_sm).fit()

print(fit.df_model)    # predictor df beyond intercept
print(fit.df_resid)    # residual df
print(fit.get_influence().hat_matrix_diag.sum()) # trace(H)
```

R: rank and residual degrees of freedom

```
X <- cbind(1, x1, x2)
n <- nrow(X)
k <- ncol(X)
r <- qr(X)$rank

print(r)
print(n - r)
print(r == k)
```

R: regression output

```
fit <- lm(y ~ x1 + x2)
df.residual(fit)
length(coef(fit))
hatvalues(fit)
sum(hatvalues(fit))
```

Output to inspect. Matrix rank, number of columns, residual degrees of freedom, omitted or unstable coefficients, and leverage sum.

Defensibility check. If $\text{rank}(X)$ is smaller than the number of columns, coefficient interpretation is not unique.

9 Multiple Linear Regression Workflow

9.1 Chapter 9 Support: Multiple Linear Regression in Practice

Purpose. Fit, interpret, predict, and compare multiple linear regression models using the Advertising example pattern.

Python: scatterplot matrix

```
import pandas as pd
import matplotlib.pyplot as plt

ad = pd.read_csv("Advertising.csv")
cols = ["Sales", "TV", "Radio", "Newspaper"]
pd.plotting.scatter_matrix(ad[cols], figsize=(8, 8), diagonal="hist")
plt.tight_layout()
plt.show()
```

Python: full model with statsmodels

```
import statsmodels.api as sm

X = ad[["TV", "Radio", "Newspaper"]]
X = sm.add_constant(X)
y = ad["Sales"]

fit = sm.OLS(y, X).fit()
print(fit.summary())
```

Python: reduced model with TV and Radio

```
X_reduced = ad[["TV", "Radio"]]
X_reduced = sm.add_constant(X_reduced)
fit_reduced = sm.OLS(y, X_reduced).fit()

print(fit_reduced.summary())
```

Python: point prediction and intervals

```
new_market = pd.DataFrame({
    "const": [1.0],
    "TV": [100],
    "Radio": [20],
    "Newspaper": [30]
})

pred = fit.get_prediction(new_market)
print(pred.summary_frame(alpha=0.05))
```

The output includes the point prediction, a confidence interval for the mean response, and a prediction interval for a new observation.

Python: model fit values

```
rss = sum(fit.resid**2)
n = fit.nobs
p = len(fit.params) - 1
rse = (rss / (n - p - 1))*0.5

print("RSS:", rss)
print("RSE:", rse)
print("R^2:", fit.rsquared)
print("Adjusted R^2:", fit.rsquared_adj)
print("Residual df:", fit.df_resid)
```

Python: residual plot

```
import matplotlib.pyplot as plt

plt.scatter(fit.fittedvalues, fit.resid)
plt.axhline(0, linestyle="--")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted")
plt.show()
```

R: full model

```
ad <- read.csv("Advertising.csv")
fit <- lm(Sales ~ TV + Radio + Newspaper, data = ad)
summary(fit)
```

R: reduced model and prediction

```
fit.reduced <- lm(Sales ~ TV + Radio, data = ad)
summary(fit.reduced)

new.market <- data.frame(TV = 100, Radio = 20, Newspaper = 30)
predict(fit, new.market, interval = "confidence")
predict(fit, new.market, interval = "prediction")
```

Defensibility check. Interpret each coefficient as an adjusted association: the expected change in the response for a one-unit change in that predictor, holding the other included predictors fixed.

10 Estimator Behavior and Residual Objects

10.1 Chapter 10 Support: Python Residual and Coefficient Objects

Purpose. Extract the objects used in the truth-plus-error and residual-geometry chapters.

Fit with statsmodels

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# y is the response Series.
# X_raw is a DataFrame containing predictor columns only.
X = sm.add_constant(X_raw)      # adds intercept column
fit = sm.OLS(y, X).fit()

print(fit.summary())
```

Extract coefficients, fitted values, and residuals

```
beta_hat = fit.params
fitted = fit.fittedvalues
resid = fit.resid
sigma_hat = np.sqrt(fit.scale)   # residual standard error
```

Build the hat matrix directly

For moderate-sized datasets, the hat matrix can be computed directly for diagnostic or instructional purposes. For routine coefficient estimation, rely on the fitted-model object or a stable solver rather than explicitly inverting $\mathbf{X}^T\mathbf{X}$.

```
Xmat = X.to_numpy()
H = Xmat @ np.linalg.inv(Xmat.T @ Xmat) @ Xmat.T
M = np.eye(Xmat.shape[0]) - H
```

For large datasets, do not build the full $n \times n$ hat matrix unless needed. The diagonal leverage values are often enough:

```
influence = fit.get_influence()
leverage = influence.hat_matrix_diag
student_resid = influence.resid_studentized_internal
```

Residual plot

```
import matplotlib.pyplot as plt
```



```
fig, ax = plt.subplots()
ax.scatter(fitted, resid)
ax.axhline(0, linestyle="--")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.set_title("Residuals vs fitted values")
plt.show()
```

Q-Q plot

```
import statsmodels.api as sm

sm.qqplot(resid, line="45")
plt.show()
```

Coefficient covariance matrix

```
coef_cov = fit.cov_params()
standard_errors = fit.bse
```

These implement the estimated version of

$$\text{Var}(\hat{\beta} \mid \mathbf{X}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}. \quad (.1)$$

10.2 Chapter 10 Support: R Residual and Coefficient Objects

Fit the model

```
fit <- lm(Y ~ X1 + X2 + X3, data = df)
summary(fit)
```

Extract fitted values and residuals

```
beta_hat <- coef(fit)
fitted_values <- fitted(fit)
residuals <- resid(fit)
sigma_hat <- summary(fit)$sigma
```

Hat values and studentized residuals

```
h <- hatvalues(fit)
r_student <- rstudent(fit)
r_standard <- rstandard(fit)
```

Diagnostic plots

```
plot(fit)
```

The default diagnostic plots include residuals versus fitted values, Q-Q plot, scale-location plot, and residuals versus leverage.

Output to inspect. Coefficients, standard errors, residual standard error, residual plots, leverage values, and studentized residuals.

11 Transformations and Linear-in-Parameters Models

11.1 Chapter 11 Support: Python Transformations

Purpose. Change the design columns while keeping the linear-in-parameters least-squares machinery.

Fit polynomial regression with statsmodels

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# df has columns y and x
x = df["x"]
X = pd.DataFrame({
    "x": x,
    "x2": x**2,
    "x3": x**3
})
X = sm.add_constant(X)

fit = sm.OLS(df["y"], X).fit()
print(fit.summary())
```

Center before polynomial terms

```
u = df["x"] - df["x"].mean()
X = pd.DataFrame({
    "u": u,
    "u2": u**2,
    "u3": u**3
})
X = sm.add_constant(X)
fit = sm.OLS(df["y"], X).fit()
```

Interaction model

```
X = pd.DataFrame({
    "x1": df["x1"],
    "x2": df["x2"],
    "x1_x2": df["x1"] * df["x2"]
})
X = sm.add_constant(X)
fit = sm.OLS(df["y"], X).fit()
```

Formula interface

```
import statsmodels.formula.api as smf

fit_poly = smf.ols("y ~ x + I(x**2) + I(x**3)", data=df).fit()
fit_int = smf.ols("y ~ x1 * x2", data=df).fit()
fit_log = smf.ols("np.log(y) ~ x", data=df).fit()
```

In the formula $y \sim x_1 * x_2$, the star expands to main effects plus interaction:

$$x_1 * x_2 \implies x_1 + x_2 + x_1 x_2. \quad (.2)$$

Prediction on a grid

```
x_grid = np.linspace(df["x"].min(), df["x"].max(), 200)
X_grid = pd.DataFrame({
    "const": 1.0,
    "x": x_grid,
    "x2": x_grid**2,
    "x3": x_grid**3
})
y_hat = fit.predict(X_grid)
```

11.2 Chapter 11 Support: R Transformations

Fit polynomial regression

```
fit_poly <- lm(y ~ x + I(x^2) + I(x^3), data = df)
summary(fit_poly)
```

Use orthogonal polynomials

```
fit_orth <- lm(y ~ poly(x, degree = 3), data = df)
summary(fit_orth)
```

Use raw polynomial columns

```
fit_raw <- lm(y ~ poly(x, degree = 3, raw = TRUE), data = df)
summary(fit_raw)
```

Interaction model

```
fit_int <- lm(y ~ x1 * x2, data = df)
summary(fit_int)
```

This expands to:

```
y ~ x1 + x2 + x1:x2
```

Transformed response

```
fit_log_y <- lm(log(y) ~ x1 + x2, data = df)
summary(fit_log_y)
```

Defensibility check. After adding transformed columns, compare residual behavior and validation error. A more complicated design matrix must improve the analysis, not only the training fit.

12 Residual Diagnostics and Model Repair

12.1 Chapter 12 Support: Python Model Diagnostics and Repair

Purpose. Diagnose leftover pattern, try targeted repairs, and compare candidate models on a common scale.

Fit a baseline model and extract residuals

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

# df has columns y, x1, x2
X = df[["x1", "x2"]]
X = sm.add_constant(X)
y = df["y"]

fit = sm.OLS(y, X).fit()
df["fitted"] = fit.fittedvalues
df["resid"] = fit.resid
print(fit.summary())
```

Residuals versus fitted values

```
fig, ax = plt.subplots()
ax.scatter(df["fitted"], df["resid"])
ax.axhline(0, linestyle="--")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.set_title("Residuals vs fitted values")
```

Normal Q-Q plot

```
fig = sm.qqplot(fit.resid, line="45")
plt.title("Normal Q-Q plot of residuals")
```

Influence and leverage

```
infl = fit.get_influence()
df["leverage"] = infl.hat_matrix_diag
df["student_resid"] = infl.resid_studentized_external
df["cooks_d"] = infl.cooks_distance[0]

# Sort largest Cook's distances
print(df.sort_values("cooks_d", ascending=False).head())
```

Try a polynomial repair

```
df["x1_c"] = df["x1"] - df["x1"].mean()
df["x1_c2"] = df["x1_c"] ** 2

X_poly = df[["x1_c", "x1_c2", "x2"]]
X_poly = sm.add_constant(X_poly)
fit_poly = sm.OLS(y, X_poly).fit()
print(fit_poly.summary())
```

Try an interaction repair

```
df["x1_x2"] = df["x1"] * df["x2"]
X_int = df[["x1", "x2", "x1_x2"]]
X_int = sm.add_constant(X_int)
fit_int = sm.OLS(y, X_int).fit()
print(fit_int.summary())
```

Try a response transformation

```
# Requires y > 0
fit_log = sm.OLS(np.log(y), X).fit()
```

```
# Predictions returned to original scale
log_pred = fit_log.predict(X)
y_pred = np.exp(log_pred)
```

Compare using validation RMSE on the same scale

When comparing repaired models, compute validation error on the response scale that matters for the analysis.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

train, test = train_test_split(df, test_size=0.25, random_state=1)

# Baseline model
X_train = sm.add_constant(train[["x1", "x2"]])
X_test = sm.add_constant(test[["x1", "x2"]])
fit_base = sm.OLS(train["y"], X_train).fit()
pred_base = fit_base.predict(X_test)

# Polynomial repair: center using the training mean only
x1_mean = train["x1"].mean()
train = train.copy()
test = test.copy()
train["x1_c"] = train["x1"] - x1_mean
test["x1_c"] = test["x1"] - x1_mean
train["x1_c2"] = train["x1_c"] ** 2
test["x1_c2"] = test["x1_c"] ** 2

X_train_poly = sm.add_constant(train[["x1_c", "x1_c2", "x2"]])
X_test_poly = sm.add_constant(test[["x1_c", "x1_c2", "x2"]])
fit_poly = sm.OLS(train["y"], X_train_poly).fit()
pred_poly = fit_poly.predict(X_test_poly)

rmse_base = np.sqrt(mean_squared_error(test["y"], pred_base))
rmse_poly = np.sqrt(mean_squared_error(test["y"], pred_poly))
print(rmse_base, rmse_poly)
```

12.2 Chapter 12 Support: R Model Diagnostics and Repair

Fit, plot residuals, and inspect diagnostics

```
fit <- lm(y ~ x1 + x2, data = df)
summary(fit)

plot(fitted(fit), resid(fit))
```

```
abline(h = 0, lty = 2)

qqnorm(resid(fit))
qqline(resid(fit))

plot(fit)    # standard diagnostic panel
```

Polynomial and interaction repairs

```
fit_poly <- lm(y ~ x1 + I(x1^2) + x2, data = df)
fit_int  <- lm(y ~ x1 * x2, data = df)

anova(fit, fit_poly)
anova(fit, fit_int)
```

Response transformation

```
fit_log <- lm(log(y) ~ x1 + x2, data = df)
summary(fit_log)
```

Influence

```
hatvalues(fit)
rstudent(fit)
cooks.distance(fit)
```

Defensibility check. A repair is justified when it addresses a visible diagnostic issue and improves the analysis without hiding an important limitation.

13 End-to-End Mini Workflow

Purpose. Use this compact workflow when starting a lab or homework analysis. It is not meant to replace the detailed recipes above; it shows the order of operations.

Python pattern

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# 1. Load
df = pd.read_csv("data/raw/my_data.csv")
```

```

# 2. Inspect and clean
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(" ", "_", regex=False)
)
print(df.info())
print(df.isna().mean().sort_values(ascending=False))

# Example cleaning decision
df.loc[df["weight_lb"] == 0, "weight_lb"] = np.nan
model_df = df.dropna(subset=["y", "x1", "x2"])

# 3. EDA
model_df[["y", "x1", "x2"]].describe()
model_df[["y", "x1", "x2"]].corr()

# 4. Split for validation
train, test = train_test_split(model_df, test_size=0.25, random_state=1)

# 5. Fit baseline MLR
X_train = sm.add_constant(train[["x1", "x2"]])
y_train = train["y"]
fit = sm.OLS(y_train, X_train).fit()
print(fit.summary())

# 6. Diagnose training residuals
train = train.copy()
train["fitted"] = fit.fittedvalues
train["resid"] = fit.resid
plt.scatter(train["fitted"], train["resid"])
plt.axhline(0, linestyle="--")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
plt.show()

# 7. Validate on held-out data
X_test = sm.add_constant(test[["x1", "x2"]])
pred = fit.predict(X_test)
rmse = np.sqrt(mean_squared_error(test["y"], pred))
print("Validation RMSE:", rmse)

```

R pattern

```

# 1. Load
df <- read.csv("data/raw/my_data.csv")

```



```

# 2. Inspect and clean
names(df) <- tolower(trimws(names(df)))
names(df) <- gsub(" ", "_", names(df))
str(df)
colMeans(is.na(df))

df$weight_lb[df$weight_lb == 0] <- NA
model_df <- df[complete.cases(df[, c("y", "x1", "x2")]), ]

# 3. EDA
summary(model_df[, c("y", "x1", "x2")])
cor(model_df[, c("y", "x1", "x2")], use = "complete.obs")

# 4. Split for validation
set.seed(1)
idx <- sample(seq_len(nrow(model_df)), size = floor(0.75 * nrow(model_df)))
train <- model_df[idx, ]
test <- model_df[-idx, ]

# 5. Fit baseline MLR
fit <- lm(y ~ x1 + x2, data = train)
summary(fit)

# 6. Diagnose training residuals
plot(fitted(fit), resid(fit))
abline(h = 0, lty = 2)

# 7. Validate on held-out data
pred <- predict(fit, newdata = test)
rmse <- sqrt(mean((test$y - pred)^2))
print(rmse)

```

Briefing sentence template. “We cleaned the data by ..., fit ... as a baseline model, checked residuals for ..., compared candidate models using ..., and concluded ... with the following limitations:”

14 Quick Troubleshooting Guide

Symptom	Likely issue	First check
Model will not run because of missing values	Required rows contain NA / NaN	Count missingness and decide drop/impute/flag
Coefficient is missing or marked not estimable	Design matrix is rank deficient	Check duplicate columns, dummy-variable trap, or perfectly related predictors
Very large standard errors	Multicollinearity or small effective sample size	Check correlations, rank, VIF-style diagnostics, and sample size
Residual plot has a curve	Mean structure may be wrong	Try transformed predictors, polynomial terms, or interactions
Residual spread grows with fitted values	Nonconstant variance	Consider response transformation or a model with different variance structure
One point dominates the model	High leverage and/or influence	Inspect leverage, studentized residuals, and Cook's distance
Train error is low but test error is high	Overfitting or data leakage	Use simpler model, proper validation, and train-only preprocessing
Correlation heatmap contains missing diagonal entries	Constant variable or insufficient variation	Check variance and remove non-informative constant columns

15 Cookbook Habits

A defensible coding workflow is not only about getting a model to run. In this course, every workflow should leave behind enough evidence that another analyst can reproduce and explain the result.

- Save the data-cleaning decisions, not just the cleaned data.
- Name derived columns so their meaning is clear.
- Keep model formulas visible.
- Extract residuals, fitted values, and leverage values when diagnostics matter.
- Compare candidate models on the same response scale.
- Keep train/test separation when estimating validation error.
- Report both the software output and the interpretation that follows from it.